

AlphaPaaS Application Hosting Standard

Reference: AHS · Version 2.4 · Effective 1 January 2026

Document owner	AlphaPaaS Cloud Broker — Platform Engineering
Approved by	AlphaInsure Group Architecture Board
Review cycle	Annual
Previous version	2.3 (March 2025)
Classification	Internal
Related standards	ARS, CKS, ASC, DCH

1. Purpose and scope

This standard defines how applications are hosted on the AlphaPaaS container platform. It covers deployment targets, workload metadata, network segmentation, service exposure, routing, and workload identity. It does not cover pod-level security controls (see the Container & Kubernetes Security Standard) or availability requirements (see the Availability & Resilience Standard).

The standard applies to all workloads deployed to AlphaPaaS, in any application zone, on any AlphaPaaS platform (AWS- or Azure-hosted).

2. Hosting targets

AHS-01	Production and pre-production workloads MUST be deployed to AlphaPaaS. Direct use of IaaS compute (EC2, Azure VMs) for application hosting is not permitted. Exceptions require a waiver from the Cloud Broker.
AHS-02	Applications MUST be deployed to the application zone that matches their exposure and data classification. The application zone is selected by the <code>alphapaas.com/app-zone-name</code> namespace label, which is set by the Cloud Broker when the subscription is created.
AHS-03	Workload manifests MUST include the mandatory platform labels: <code>app</code> , <code>app-id</code> , <code>component</code> , <code>part-of</code> , <code>tier</code> , and <code>owner</code> . The <code>app-id</code> value MUST match the identifier issued in the application intake form.

3. Service exposure

AlphaPaaS supports four exposure patterns, each served by a dedicated ingress router. The pattern is selected by applying the router label to the Route resource:

Router label	Source of traffic	Intended use
<code>router=internal</code>	AlphaInsure internal network	Internal APIs not exposed beyond AlphaPaaS
<code>router=irp</code>	Internal, via approved reverse proxy	Internal APIs behind a controlled gateway
<code>router=external</code>	Internet, via AlphaInsure WAF	Public APIs protected by on-premise WAF
<code>router=waf</code>	Internet, via SaaS WAF / CDN	Public-facing services protected by Silverline / CloudFront

- AHS-04** Every Route resource **MUST** include an explicit router label. Routes without the label are silently defaulted to `router=internal`, which may not reflect the team's intent and is a frequent source of exposure mismatches.
- AHS-05** The router label applied in production **MUST** match the exposure declared in the application intake form. A mismatch (for example, a Route labelled `router=external` on an application declared as `internal-only`) is a material security finding.
- AHS-06** Routes exposing Restricted or Confidential data **MUST** use TLS. Allowed termination modes are `edge`, `passthrough`, and `reencrypt`. Plain HTTP routes for Restricted data are prohibited.
- AHS-07** For TLS-terminating routes (`edge`, `reencrypt`), `insecureEdgeTerminationPolicy` **MUST** be set to `Redirect`. The value `Allow` is prohibited as it permits plaintext downgrades.
- AHS-08** Routes targeting more than one pod **MUST** set the `haproxy.router.openshift.io/balance` annotation. Accepted values are `roundrobin` or `leastconn`. The source algorithm is prohibited, as it produces uneven distribution behind the load balancer.

NOTE — Why source balancing is prohibited

The AlphaPaaS ingress load balancer presents only one source IP per availability zone to the OpenShift routers. Using the source algorithm will pin traffic to as few as three pods regardless of how many replicas are running, defeating horizontal scaling.

4. Workload identity

Every workload runs under a Kubernetes ServiceAccount. The ServiceAccount determines which Kubernetes API calls the workload can make, and — for workloads federated with a cloud provider — which AWS IAM role or Azure managed identity the workload may assume.

- AHS-09** Workloads **MUST** run under a dedicated ServiceAccount. The default ServiceAccount in a namespace **MUST NOT** be used for application workloads. This aligns with OCP CIS benchmark 5.1.5.
- AHS-10** Every workload **MUST** set `spec.automountServiceAccountToken` explicitly (either `true` or `false`). Relying on the Kubernetes default is prohibited. Workloads that do not call the Kubernetes API **MUST** set the value to `false`. This aligns with OCP CIS benchmark 5.1.6.
- AHS-11** Workloads requiring access to cloud provider APIs (AWS services, Azure services) **MUST** assume a role via workload identity federation. Long-lived access keys or service principal passwords stored in secrets are prohibited.

5. Network segmentation

AlphaPaaS namespaces are created with a baseline set of NetworkPolicies that apply default-deny for cross-namespace traffic and allow the minimum required intra-cluster traffic. These baseline policies cannot be deleted or weakened. Application teams must add further NetworkPolicies to permit the specific flows their workloads require.

- AHS-12** Every application namespace **MUST** contain at least one team-managed NetworkPolicy. Relying solely on the AlphaPaaS baseline is permitted only for workloads that have no internal dependencies, which is rare.

- AHS-13** Ingress NetworkPolicies MUST be specific: they MUST target pods by label selector and allow traffic only from named namespaces or pod selectors. NetworkPolicies with empty podSelectors that allow from 0.0.0.0/0 are prohibited.
- AHS-14** Egress traffic to the public internet MUST be restricted using egress NetworkPolicies (EgressNetworkPolicy or EgressFirewall). Workloads that require broad internet access MUST document the reason in their runbook.
- AHS-15** Egress to cloud provider APIs (EC2, Azure ARM, etc.) MUST route via VPC endpoints or Azure Private Link wherever supported by the target service. Traffic to approved services MUST NOT leave the AlphaPaaS network fabric.

6. Cross-CSP connectivity

AlphaPaaS supports workloads that consume services from a cloud provider other than the one hosting the AlphaPaaS cluster (for example, a cluster on AWS consuming a database on Azure). Cross-CSP traffic flows through the AlphaInsure vWAN Hub and inter-CSP interconnects.

- AHS-16** Cross-CSP application traffic MUST route over private connectivity (vWAN Hub, Transit Gateway, Private Link / Private Endpoint). Cross-CSP traffic over the public internet is prohibited.
- AHS-17** Cross-CSP service consumers MUST resolve target service FQDNs via AlphaPaaS private DNS. Consumers that bypass private DNS and connect to public FQDNs will incur data egress charges and break the private-connectivity guarantee.

IMPORTANT — Common cross-CSP pitfall

A pod on AWS-hosted AlphaPaaS consuming an Azure PostgreSQL Flexible Server must resolve the Postgres hostname to its private endpoint IP via the AlphaPaaS private DNS zone. If the pod uses the public Azure FQDN, the connection will succeed but will route over the public internet, bypassing the Private Endpoint. This is a breach of AHS-16.

7. Certificates

- AHS-18** Routes using TLS MUST present a certificate issued by either the AlphaPaaS PKI, the AlphaPaaS service CA (for internal mTLS), or an approved public CA. Self-signed certificates are prohibited outside developer sandboxes.
- AHS-19** Certificates MUST be renewed before expiry. Route owners MUST ensure that alerting is in place at least 30 days before expiry.

8. Exceptions

Exceptions are granted by the Cloud Broker via the standard waiver process. Waivers for clauses AHS-01, AHS-05, AHS-06, AHS-11, and AHS-16 require additional sign-off from Group Security.

9. Related standards

- **ARS** — AlphaPaaS Availability & Resilience Standard
- **CKS** — AlphaPaaS Container & Kubernetes Security Standard
- **ASC** — AlphaPaaS Approved Services Catalog
- **DCH** — AlphaPaaS Data Classification & Handling Standard